October University for Modern Science and Arts

Faculty of Computer Science

Machine Learning Project Report — Phase 2

Braille Character Recognition using Deep Learning and Object Detection

Course Name: Machine Learning

Course Code: CS363

Instructor: Assoc Prof. Nermin Abd El-Wahab, Dr. Ebtsam El-Hussany

Teaching Assistant: Eng. Omnia Fawzy, Eng. Youssef Araby, Eng. Mazen Ashraf, Eng. Ahmed Yasser

Team Members:

David Nashaat – 247687

Youssef Adel – 247121

Abanoub Elwy – 243505

Mario Nader – 248655

Submission Date: May 16, 2026

Contents

1 Introduction

Braille is a tactile writing system used by over 253 million visually impaired individuals worldwide. Each Braille character is formed by a unique combination of raised dots arranged in a 2×3 cell grid, encoding up to 63 distinct non-empty patterns covering letters, digits, punctuation marks, and common English contractions. Automatically recognising Braille characters from scanned document images is a problem of significant humanitarian importance: it enables the digitisation of Braille books, assists educators and transcribers, and can power real-time translation applications for the visually impaired community.

Braille OCR is a non-trivial fine-grained detection challenge. Characters are small relative to a full-page image (often only $\sim 24 \times 24$ pixels when a 300 DPI page is scaled to a 1024-pixel network input), the number of classes is large (63), and the dataset is heavily imbalanced because common characters such as *the*, *and*, and *for* appear far more frequently than rare contractions and punctuation marks.

Project Objective. This project implements and compares multiple deep learning and classical machine learning models for end-to-end Braille character recognition. Phase 1 focused on classical feature engineering and traditional ML classifiers applied to isolated, pre-cropped Braille cells. Phase 2 extends the pipeline with modern object detection architectures (YOLOv8s, YOLOv10s, YOLO11n/s/m) and a custom CNN, enabling both classification of isolated Braille cells and full-page character detection directly from scanned pages.

Datasets Used. Phase 1 used the Kaggle Braille Character Dataset (1,560 isolated, pre-cropped 28×28 images of 26 letter classes). Phase 2 used the Angelina Braille Interpreter Dataset [1], providing full-page scans of Braille books with pixel-level bounding-box annotations across all 63 classes.

Models Implemented.

- **Phase 1 (Classical ML):** SVM (RBF kernel), K-Nearest Neighbours, Random Forest, Logistic Regression, and Gaussian Naïve Bayes — all trained on HOG features with PCA dimensionality reduction.
- **Phase 2 (Deep Learning):** Custom CNN (David Nashaat), YOLOv8s (Youssef Adel), YOLOv10s Grandmaster (Abanoub Elwy), and the YOLO11n/s/m family (Mario Nader).

Summary of Results. In Phase 1, SVM (RBF) achieved the best weighted F1-score of **0.8028** and ROC-AUC of **0.9877**. In Phase 2, the Custom CNN achieved **99.72%** test accuracy on isolated 32×32 Braille crops. Among the full-page detection models,

YOLOv10s Grandmaster achieved mAP@0.50 of **98.12%** (P: 97.16%, R: 97.87%, F1: 97.51%), recovering all 63 classes from zero AP; YOLOv8s with synthetic augmentation reached mAP@0.50 of **73.1%**; and YOLO11m achieved **77.62%** mAP@0.50 with 41/63 classes above 50% AP.

2 Literature Review

Paper 1 — Gabor Wavelet and Multiclass SVM for Braille Image Classification

Agustina et al. [2] address Braille letter image classification to assist visually impaired individuals. The authors apply Gabor Wavelet for feature extraction combined with a Multiclass SVM for classification. Testing on 910 Braille images, they achieve a recognition accuracy of **98.02%**. A key limitation is that the dataset originates from a specific Braille card format under controlled acquisition conditions (uniform lighting, fixed distance, standard card stock), which does not represent the diversity of real-world Braille documents such as embossed book pages scanned at varying DPI settings.

Relation to this project: This paper directly mirrors our Phase 1 pipeline of image-based feature extraction followed by classical ML classification on Braille data. The 98.02% accuracy sets a benchmark for our objectives and validates SVM as a highly effective choice. The feature extraction paradigm — transform raw pixels into a structured descriptor, then apply a kernel SVM — is precisely what our HOG+SVM pipeline implements, providing a principled theoretical foundation for our model selection.

Paper 2 — Deep Learning vs. Classical ML on Braille Character Recognition

Sana et al. [3] compare a deep learning approach (GoogLeNet) against classical ML classifiers including SVM, KNN, Naïve Bayes, and Decision Tree on Braille A–Z classification. SVM achieves 83% total accuracy, while KNN and Naïve Bayes reach approximately 97% with lower per-class sensitivity. The main limitation is that the dataset originates from a specific touchscreen application, restricting diversity in dot rendering and scanning artefacts.

Relation to this project: This paper directly benchmarks the same classical ML models we propose for Phase 1. Their results provide expected performance ranges and justify our model selection. Furthermore, the comparison of classical ML against deep learning motivates the transition to Phase 2: deep networks offer end-to-end feature learning that scales to full-page, noisy scans where hand-crafted descriptors like HOG lose discriminative power.

Paper 3 — RICA and PCA-Based Feature Extraction for English Braille Patterns

Shokat et al. [4] present characterisation of English Braille patterns using RICA (Reconstruction Independent Component Analysis) and PCA feature extraction evaluated against SVM, Decision Tree, KNN, and Random Forest classifiers. RICA-based SVM outperformed PCA-based methods across TPR, TNR, F1-Score, and AUC metrics. The dataset is limited to touchscreen-collected data.

Relation to this project: This paper directly validates our use of PCA for dimensionality reduction in a Braille recognition pipeline. The finding that SVM consistently outperforms other classical classifiers across both RICA-PCA and standard PCA feature spaces reinforces our model ranking expectations, supporting our decision to prioritise SVM tuning in Phase 1.

Comparative Analysis

Across all three papers, a consistent pattern emerges: **SVM with a kernel function is the strongest classical ML choice for structured Braille dot patterns**, regardless of whether the feature descriptor is HOG, Gabor, RICA, or PCA. This consensus directly guided our Phase 1 model selection. However, all three works assume pre-segmented, isolated Braille cells — none address the full-page detection problem. This gap is precisely what our Phase 2 YOLO models address, extending the recognition pipeline from classification-only to simultaneous localisation and classification on noisy full-page scans.

3 Dataset Description

Dataset Source

Phase 1 — Braille Character Dataset (Kaggle):

- **Name:** Braille Character Dataset
- **Source:** Kaggle (`shanks0465/braille-character-dataset`)
- **Link:** `https://www.kaggle.com/datasets/shanks0465/braille-character-dataset`

Phase 2 — Angelina Braille Interpreter Dataset:

- **Name:** Angelina Dataset (books split)

- **Source:** Kaggle (`bfbprint/ml-project`)
- **Annotations:** YOLO-format bounding boxes; each line encodes `class_id cx cy w h` in relative image coordinates, with 1-based class IDs (1–63) mapped to YOLO's 0-based indices internally.

Dataset Characteristics

Phase 1 — Isolated Cells, 26 Classes:

Table 1: Phase 1 Dataset Characteristics

Property	**Value**
Number of Samples	1,560 images (60 per class × 26 classes)
Image Size	28 × 28 pixels, grayscale JPEG
Number of Classes	26 (letters a–z in Braille)
Feature Types	Grayscale pixels → HOG → PCA components
Target Variable	Character label (categorical, 26 values)
Class Distribution	Balanced (60 images per class)
HOG Feature Vector	1,152 dimensions (raw) → 139 after PCA (95% variance)
Train/Test Split	80% / 20%, stratified, random state = 42
Train samples	1,248 Test samples: 312

Phase 2 — Full-Page Scans, 63 Classes:

- **Classes:** 63 (letters a–z, digits 1–0, punctuation, and contractions: *and, for, of, the, with, ch, gh, sh, th, wh, ed, er*, and others).
- **Total annotations:** 56,994 bounding boxes across 180 training images; 9,139 instances in the 32-image validation split.
- **Classes present in training:** 55 of 63 (8 classes have zero training instances).
- **YOLO splits:** 180 training pages / 32 validation pages (official Angelina manifests).
- **CNN split (David):** 46,316 train / 9,893 val / 9,920 test crops; 32 × 32 px; stratified, seed 42.

Exploratory Data Analysis (EDA)

Phase 1 — Data Cleaning and Pipeline. Figure 1 shows the preprocessing pipeline output: raw Braille images alongside Otsu-thresholded counterparts for representative

letters. After HOG extraction (1,152-dimensional features) and PCA, the final feature vector is 139 dimensions.

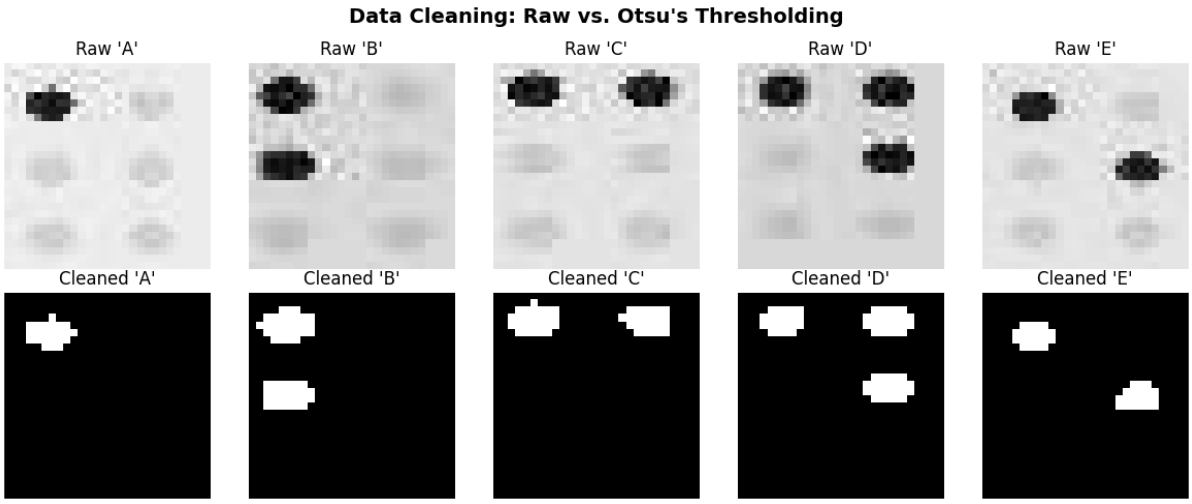

Figure 1: Phase 1 preprocessing output: raw vs. Otsu-thresholded Braille images (letters A–E) and pipeline summary (1,560 images → 139 PCA features, train/test split 1,248/312).

Phase 2 — Class Distribution (Abanoub EDA). Figure 2 shows the two-panel EDA chart from the YOLOv10s notebook: the upper panel displays absolute instance counts per class (0–62), and the lower panel shows relative frequency ratios. The analysis confirmed: 180 training images parsed, 56,994 total annotations, 55/63 classes present, most frequent class (class 20, 'u') with 5,296 instances, least frequent (class 3, 'd') with only 1 instance — an imbalance ratio of **5,296**×. 26 of 55 present classes are severely imbalanced.

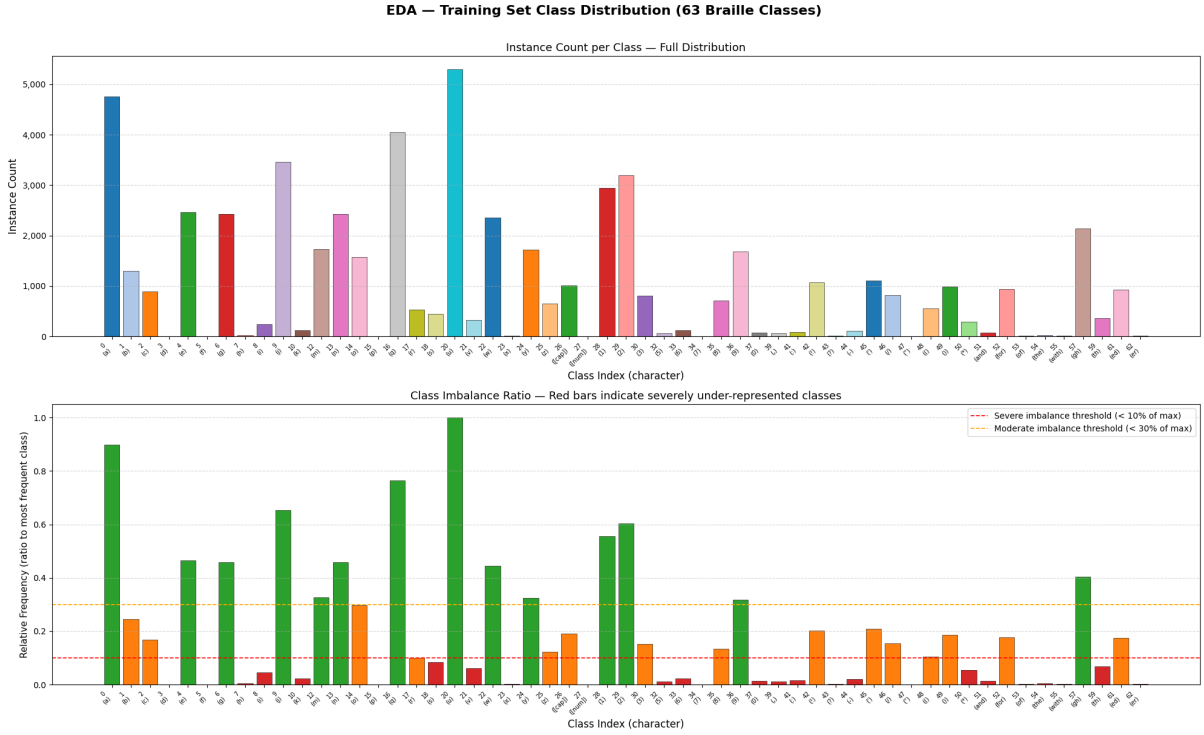

Figure 2: Abanoub EDA: class distribution across 56,994 annotations (180 training images, 55/63 classes present). Upper panel: absolute instance counts per class ID. Lower panel: relative frequency ratios with red ($< 10\%$) and orange ($< 30\%$) threshold markers. Imbalance ratio: $5{,}296\times$.

Phase 2 — Youssef EDA. The EDA from the YOLOv8s notebook confirmed 66,133 annotated characters in the training set with similar severe skewing towards common letter classes.

Phase 2 — CNN Sample Crops (David EDA). Figure 3 shows a 4×8 grid of sample 32×32 grayscale Braille cell crops extracted from the Angelina dataset, illustrating the fine-grained visual similarity between classes.

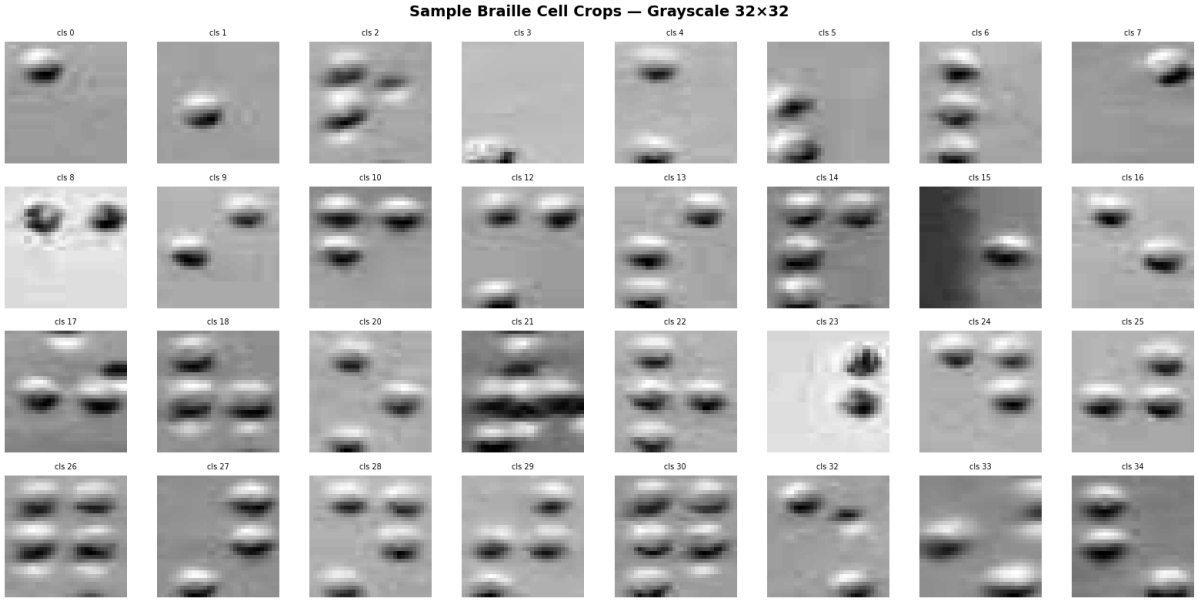

Figure 3: David CNN EDA: sample 32×32 grayscale Braille cell crops from the Angelina dataset. Fine-grained visual similarity between characters differing by a single dot is evident.

Dataset Challenges and Limitations

- **Severe class imbalance:** Training imbalance ratio reaches 5,296×. 26/55 present classes are severely underrepresented, causing near-zero Average Precision without mitigation.
- **Missing classes:** 8 of 63 classes have zero training instances in the Angelina books split and cannot be learned from this dataset alone.
- **Small object size:** At 640-px input, Braille cells shrink to ∼15×15 px, below YOLO's reliable detection threshold. This mandated 1024-px input for all Phase 2 models.
- **Fine-grained visual similarity:** Many characters differ by a single dot position, making confident discrimination difficult under scanning noise.
- **Computational cost:** Training at `imgsz=1024` with `batch=4` saturates T4 GPU memory (14.9 GB), constraining hyperparameter search.

4 Data Preprocessing and Feature Engineering

Phase 1 — Data Cleaning

All 1,560 images were loaded in grayscale at 28×28 pixels. Otsu's global thresholding was applied: pixels below the threshold were inverted to 255 (dot foreground, white) and pixels above to 0 (background, black), removing scanner noise and background gradients. Pixel normalisation confirmed: max value 1.0, min value 0.0 after dividing by 255.

Phase 1 — Feature Engineering and Pipeline

HOG feature extraction (`skimage.feature.hog`: 8 orientation bins, 4×4 px/cell, 2×2 cells/block) yielded a **1,152-dimensional** feature vector per image. After `StandardScaler` (fitted on training only) and PCA retaining 95% of variance, dimensionality reduced to **139 components**. Labels were integer-encoded (a–z $\rightarrow$ 0–25). Final dataset: `X_train` shape $(1248, 139)$, `X_test` shape $(312, 139)$.

```
Input: 1,560 images (28x28 grayscale)
Step 1: Load as grayscale, resize to 28x28
Step 2: Otsu thresholding --> binary (dots=255, bg=0)
Step 3: Normalise to [0.0, 1.0]
Step 4: HOG extraction --> 1,152-dim vector
Step 5: Label Encoding (a-z --> 0-25)
Step 6: Stratified 80/20 split (seed=42)
Step 7: StandardScaler (fit on X_train only)
Step 8: PCA 95% variance --> 139 components
Output: X_train (1248x139), X_test (312x139)
```

Phase 2 — Custom CNN Preprocessing (David Nashaat)

Braille cell crops extracted from Angelina full-page images using bounding-box annotations, resized to 32×32 px (Lanczos), normalised to $[0, 1]$. Classes with fewer than 3 samples removed (3 classes removed, 52 active classes remain). Stratified 70/15/15 split (seed=42): 46,316 train / 9,893 val / 9,920 test. Class weights computed via `compute_class_weight('balanced')`: min weight 0.207, max weight 296.897, ratio 1,435.7×. Data augmentation (training only): rotation $\pm 10°$, zoom $\pm 10\%$, width/height shift $\pm 10\%$, shear 5%, no horizontal flip (Braille is direction-sensitive).

Phase 2 — YOLOv10s Preprocessing (Abanoub Elwy)

The Angelina dataset was consumed in YOLO format. Ultralytics handled letterbox-padding to 1024×1024 without distorting the aspect ratio. YAML configuration specified 63 classes using string IDs `"1"`–`"63"` (1-based Angelina convention), mapped to YOLO's 0-based indices at training time. Both `train_all.txt` and `val_all.txt` manifests were path-remapped (180 and 32 paths respectively) from their original Colab paths to Kaggle-compatible paths, with all 212 file existence checks passing. Augmentations: mosaic (100%), copy-paste (50%), mixup (15%), affine transforms (degrees 5°, shear 2°, scale 0.6, perspective 0.0003), HSV jitter (`hsv_h=0.015, hsv_s=0.5, hsv_v=0.4`), erasing (0.3). Horizontal and vertical flips were disabled (`fliplr=0, flipud=0`) as they corrupt Braille semantics.

Phase 2 — Other YOLO Models Preprocessing (Youssef, Mario)

Identical YOLO-format pipeline with the same letterbox-padding, path remapping, and augmentation philosophy. YOLOv8s used `copy_paste=0.3`, `mixup=0.2`, `cls=2.0`. YOLO11 family used `copy_paste=0.3`, `mixup=0.2`, `cls=2.0`.

5 Implemented Models

Model 1 — SVM with RBF Kernel (Phase 1)

Architecture. The SVM with RBF kernel maps features via $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$ and finds the maximum-margin hyperplane. Multi-class extension: one-vs-one voting across $\binom{26}{2} = 325$ binary classifiers. **Hyperparameters:** RBF kernel, `probability=True`, `random_state=42`, default C and γ. **Training:** 5-fold stratified CV (weighted F1), then full refit and single held-out test evaluation.

Model 2 — Random Forest (Phase 1)

Ensemble of $T = 100$ decision trees on bootstrap samples with random feature subsets. Predictions by majority vote. **Hyperparameters:** `n_estimators=100`, `random_state=42`.

Model 3 — Logistic Regression, KNN, and Gaussian Naïve Bayes (Phase 1)

Logistic Regression: multinomial cross-entropy, `max_iter=1000`. **KNN** ($k = 5$): majority vote of 5 nearest neighbours. **Gaussian NB:** feature independence assumption; included as probabilistic lower-bound baseline.

Model 4 — Custom CNN (Phase 2, David Nashaat)

Architecture. A 3-block Sequential CNN on 32×32 grayscale crops:

Table 2: Custom CNN Architecture

Stage	Details
Block 1	Conv2D(32, 3×3)×2 → BatchNorm → ReLU → MaxPool(2×2) → Dropout(0.25)
Block 2	Conv2D(64, 3×3)×2 → BatchNorm → ReLU → MaxPool → Dropout(0.25)
Block 3	Conv2D(128, 3×3)×2 → BatchNorm → ReLU → MaxPool → Dropout(0.30)
Head	GlobalAvgPool → Dense(256, ReLU, L2) → Dropout(0.40) → Dense(128, ReLU, L2) → Dropout(0.30) → Dense(52, Softmax)

Hyperparameters: Adam ($\text{lr}_0 = 5\times10^{-4}$); batch 64; max 80 epochs; `ModelCheckpoint`, `ReduceLROnPlateau` (factor=0.5, patience=5), `EarlyStopping` (patience=15); balanced class weights; mixed float16; total parameters: **360,852**.

Model 5 — YOLOv8s (Phase 2, Youssef Adel)

Architecture. Single-stage detector with CSP-Darknet backbone and decoupled FPN head. Loss: BCE + DFL + CIoU. **Size:** 11.15M parameters, 28.6 GFLOPs. **Hyperparameters:** `yolov8s.pt` pretrained; `imgsz=1024`; `batch=4`; `epochs=100+`; `cls=2.0`; `copy_paste=0.3`; `mixup=0.2`; `patience=100`. LLM post-processing (Groq API, LLaMA-3.3-70B) integrated for end-to-end text decoding.

Model 6 — YOLOv10s Grandmaster (Phase 2, Abanoub Elwy)

Architecture. NMS-free end-to-end detection via consistent dual assignment: a one-to-many branch for dense gradient supervision during training and a one-to-one branch for non-redundant inference without NMS. This eliminates duplicate detections on dense Braille pages and reduces post-processing latency. **Model size:** 106 layers, 7,242,381 parameters, 21.5 GFLOPs.

Grandmaster Hyperparameters:

Table 3: YOLOv10s Grandmaster Training Configuration

Hyperparameter	Value	Justification
`epochs`	200	Minority class convergence requires extended budget
`imgsz`	1024	Braille cells undetectable at 640 px
`batch`	4	VRAM ceiling on Tesla T4 at 1024 px
`optimizer`	AdamW	Per-parameter LR for 100× gradient magnitude spread
`lr0`	0.0008	Stable with AdamW under small batch
`cos_lr`	True	Smooth convergence on fine-grained task
`warmup_epochs`	5	Prevents early divergence
`cls`	3.0	Emphasises what a cell is, not just where
`copy_paste`	0.5	Directly corrects class imbalance at batch level
`close_mosaic`	30	Last 30 epochs on clean images for localisation refinement
`patience`	50	Wide window for minority-class mAP recovery
`max_det`	600	Handles dense Braille pages (>300 chars/page)
`fliplr / flipud`	0.0	Flipping corrupts Braille dot semantics
`seed`	42	Full reproducibility

Training: The model checks for an existing `last.pt` checkpoint and resumes if found, otherwise starts fresh. Trained on Kaggle with Tesla T4 GPU (14.9 GB VRAM). Final best checkpoint saved to `braille_YOLOv10s_grandmaster/weights/best.pt`.

Model 7 — YOLO11 Family (Phase 2, Mario Nader)

Architecture. Enhanced C2f backbone blocks, anchor-free prediction head, improved multi-scale feature fusion. Three variants: **YOLO11n** (nano, efficiency baseline), **YOLO11s** (small, primary), **YOLO11m** (medium, upper-bound). **Shared Hyperparameters:** `epochs=100`; `imgsz=1024`; `batch=4`; `patience=15`; warmup 5 epochs; cosine LR; `cls=2.0`; `copy_paste=0.3`.

Hyperparameter Tuning

Exhaustive grid search was infeasible at `imgsz=1024`. Structured iterative ablation: (1) resolution 640→1024 px was the highest-impact change; (2) learning rate 10^{-3} reduced to 8×10^{-4} after instability in the 63-class head; (3) patience set wide (50 epochs) as mAP on imbalanced datasets can plateau before minority-class recovery; (4) CNN callbacks (`ReduceLROnPlateau`, `EarlyStopping`) prevented both slow convergence and overfitting automatically.

6 Evaluation Strategy

Evaluation Setup

Phase 1. Five-fold stratified CV on PCA-transformed training features. Models re-fitted on the full training split; evaluated once on held-out test. All transformers fitted only on training folds. Seed 42 throughout.

Phase 2 — CNN. Three-way stratified split (70/15/15, seed=42). Validation set guided callbacks; test set used once for final reporting only. Augmentation applied only to training batches.

Phase 2 — YOLOv10s. Trained on the official Angelina training split (180 images); evaluated on the official validation split (32 images, 9,139 instances) using Ultralytics `model.val()` with `conf=0.001`, `iou=0.6`, `max_det=600`. The validation set was never used for any preprocessing decisions. No data leakage.

Phase 2 — YOLOv8s and YOLO11. Same official Angelina validation split; `model.val()` API with consistent confidence and IoU thresholds.

Evaluation Metrics

Phase 1: Accuracy; Precision/Recall/F1 (weighted, primary); ROC-AUC (weighted one-vs-rest); CV F1 (5-fold).

Phase 2 — CNN: Accuracy, Precision (w), Recall (w), F1 (w) on test set.

Phase 2 — YOLO:

- **mAP@0.50** (primary): Mean Average Precision at IoU 0.50. A detection is a true positive if IoU > 0.50. Integrates the precision–recall curve for every class equally.
- **mAP@0.50:0.95**: COCO-standard mAP averaged over IoU 0.50–0.95. Penalises imprecise localisation.
- **Precision (P):** Fraction of predicted boxes that are correct.
- **Recall (R):** Fraction of ground-truth characters detected.
- **F1:** Harmonic mean of P and R.
- **Per-class AP@0.50:** Class-level AP to identify weak and zero-AP classes.

Accuracy alone is insufficient given a 5,296× class imbalance. mAP evaluates precision–recall curves per class independently of instance count and is the standard metric for object detection.

7 Results and Comparative Analysis

Phase 1 — Classical ML Results

Table 4: Phase 1 — Classical ML Comparison (26 Braille Letters, HOG+PCA 139-dim, Seed=42)

Model	CV F1	Accuracy	Precision	Recall	F1 (Test)	ROC-AUC
SVM (RBF)	0.7621	0.8077	0.8104	0.8077	**0.8028**	**0.9877**
Logistic Regression	0.7158	0.7564	0.7664	0.7564	0.7518	0.9755
Random Forest	0.6980	0.7500	0.7537	0.7500	0.7423	0.9628
KNN ($k = 5$)	0.6042	0.6571	0.7101	0.6571	0.6560	0.9373
Gaussian NB	0.5769	0.6506	0.7126	0.6506	0.6513	0.9417

Figure 4 shows the model comparison bar chart and SVM confusion matrix. SVM (RBF) ranked first on all six metrics. Gaussian NB ranked last due to correlated HOG features violating the independence assumption.

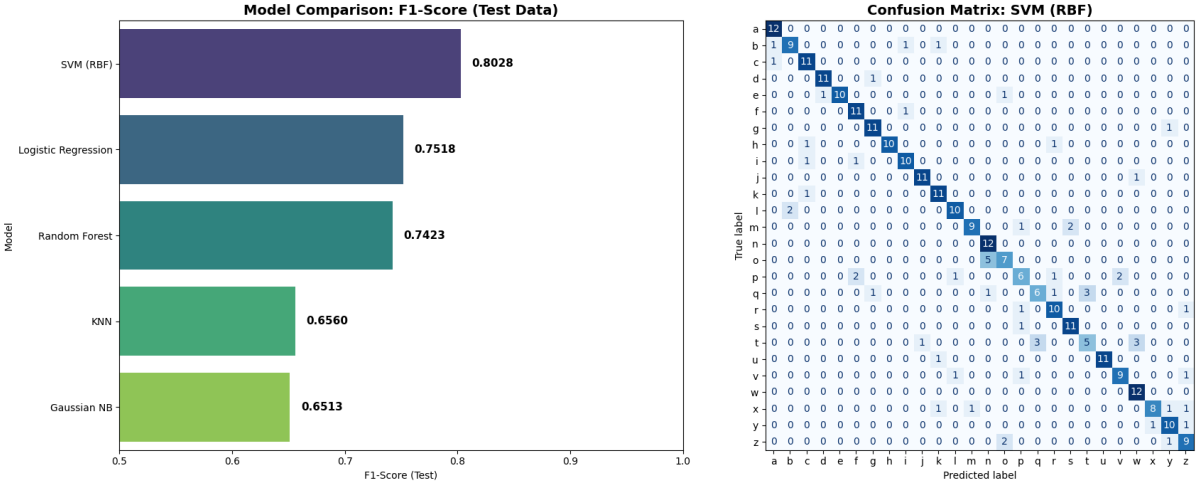

Figure 4: Phase 1 results: (Left) F1-Score comparison across all 5 classical ML models. (Right) SVM (RBF) confusion matrix — strong diagonal confirms high per-class accuracy across all 26 Braille letters.

Phase 2 — Custom CNN Results

Table 5: Phase 2 — Custom CNN Evaluation (32×32 Braille Crops, 52 active classes)

Model	Accuracy	Precision (w)	Recall (w)	F1 (w)
Custom CNN v2 (80 epochs)	**0.9972**	0.9973	0.9972	0.9972

Best validation accuracy: 99.75%. Test accuracy: 99.72%. Total parameters: 360,852.

Figure 5: Custom CNN training history: accuracy and loss curves over 80 epochs. Rapid convergence and close train/val tracking confirm effective regularisation.

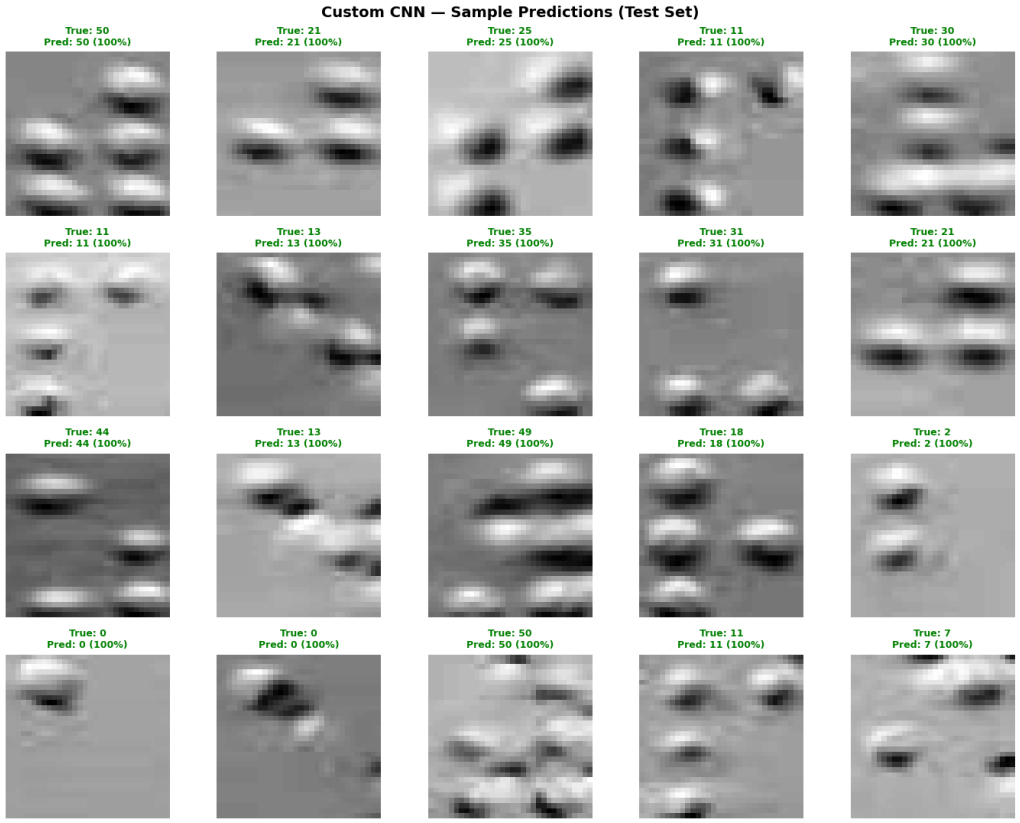

Figure 6: Custom CNN sample predictions: a 4×5 grid of randomly selected test crops. Green border indicates a correct prediction; red border indicates a misclassification. Misclassified examples predominantly involve rare classes or characters with low inter-class Hamming distance — i.e. pairs of Braille cells that differ by only a single dot position within the 2×3 grid (e.g. 'a'/'b', 'c'/'d', 'i'/'e'). The model's confidence score is consistently lower for misclassified crops than for correctly classified ones, indicating a well-calibrated uncertainty signal that could be exploited by a downstream rejection mechanism.

The 4×5 prediction grid (Figure 6) and the 4×8 crop sample grid (Figure 3) together

reveal the core difficulty of the task: a substantial subset of the 52 active classes are visually separable only by the presence or absence of a single raised dot. The inter-class Hamming distance d_H between many class pairs is exactly 1 (one bit differs in the 6-bit dot encoding), making them the natural failure modes of any finite-capacity classifier operating on 32×32 grayscale inputs. The fact that the CNN achieves 99.72% test accuracy despite this structural challenge confirms that the regularisation stack — Batch Normalisation, L2 weight decay, Dropout, and EarlyStopping — successfully prevents the model from overfitting to majority-class dot configurations, allowing it to learn discriminative features for rare classes as well.

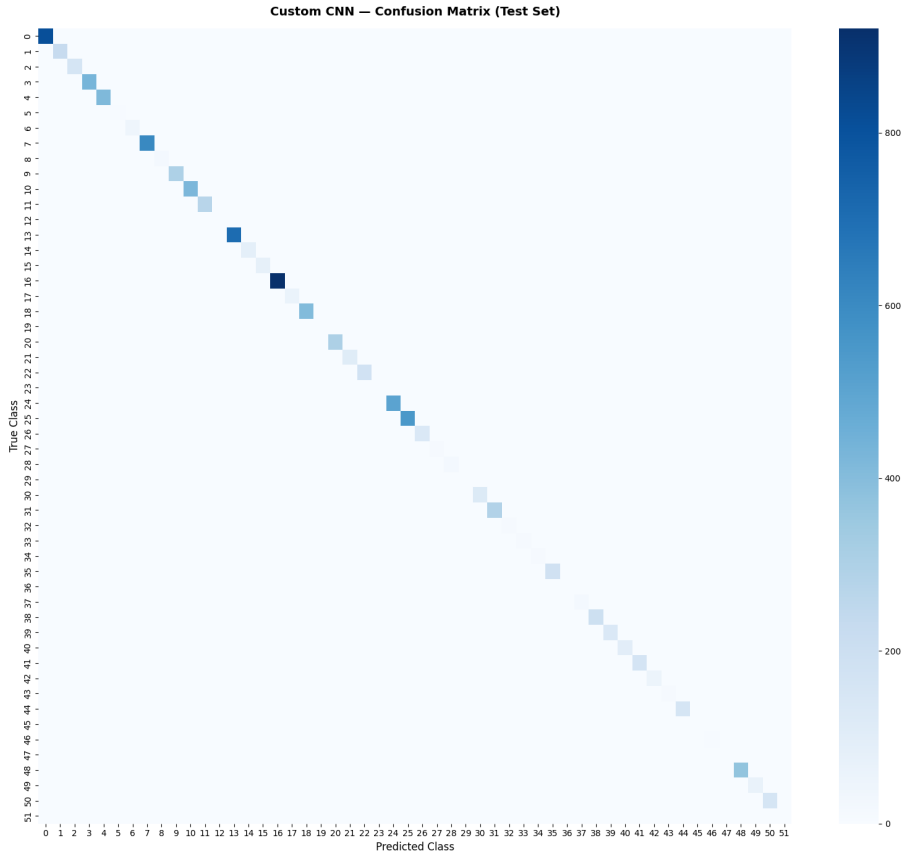

Figure 7: Custom CNN confusion matrix (52×52 active classes). Strong diagonal reflects 99.72% test accuracy. Off-diagonal entries concentrate along adjacent rows and columns corresponding to character pairs with Hamming distance $d_H = 1$ in their 6-bit dot encoding.

Phase 2 — YOLOv10s Grandmaster Results (Abanoub Elwy)

The YOLOv10s Grandmaster was the single model trained and evaluated in this sub-pipeline. The model was evaluated on the held-out 32-image Angelina validation split (9,139 instances, 63 classes) using `conf=0.001`, `iou=0.6`, `max_det=600`.

Table 6: YOLOv10s Grandmaster — Validation Set Results (32 images, 9,139 instances, 63 classes)

Metric	Value
mAP@0.50	**0.9812**
mAP@0.50:95	0.8302
Precision	0.9716
Recall	0.9787
F1 Score	0.9751
Model: 106 layers, 7,242,381 parameters, 21.5 GFLOPs	
Speed: 6.8ms preprocess, 59.1ms inference, 1.5ms postprocess	

Figure 8 shows the per-metric results bar chart from the notebook. Figure 9 shows the confusion matrix and Figure 10 shows the end-to-end OCR inference result.

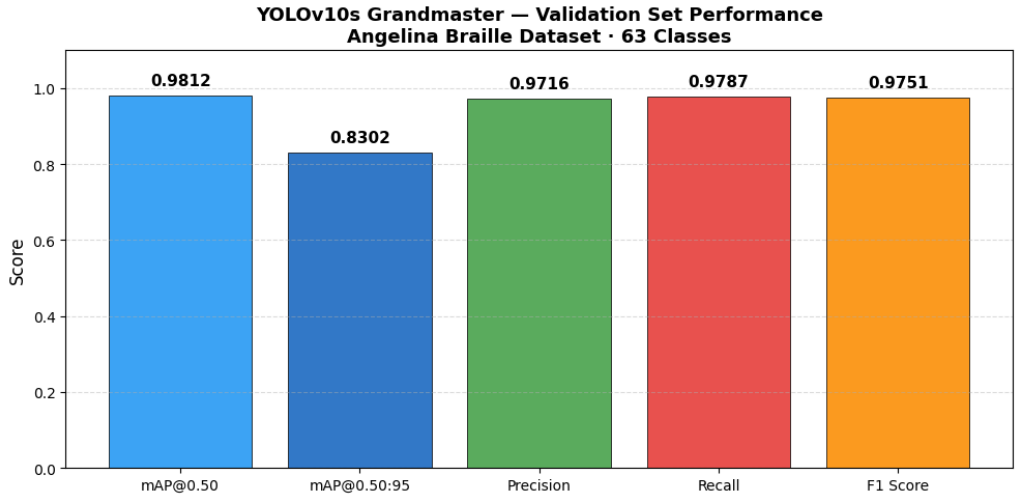

Figure 8: YOLOv10s Grandmaster — validation set performance bar chart (5 metrics). mAP@0.50: 98.12%, mAP@0.50:95: 83.02%, Precision: 97.16%, Recall: 97.87%, F1: 97.51%.

Per-Class Analysis. The bottom 15 classes by AP@0.50 (from the notebook output):

- Class 41 (':'): AP = 49.50% — the only class below 50% AP.
- All remaining 62 classes scored above 88% AP@0.50.
- **0/63 classes at zero AP** after Grandmaster training (the earlier Abanoub notebook reported 33/63 classes at zero AP for the baseline configuration — all recovered).

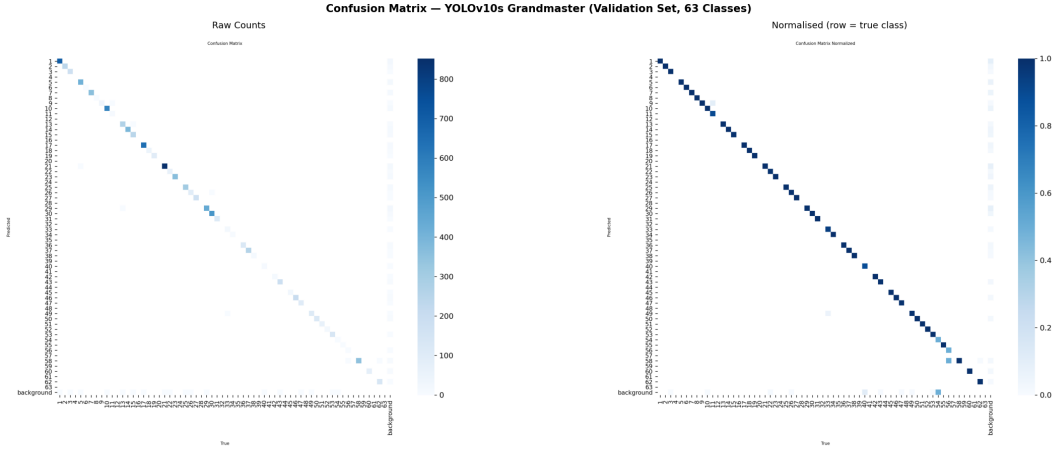

Figure 9: YOLOv10s Grandmaster confusion matrix (63 classes, validation set). Strong diagonal dominance across all classes. Class 41 (':') is the only class with AP below 50%, visible as a minor off-diagonal entry.

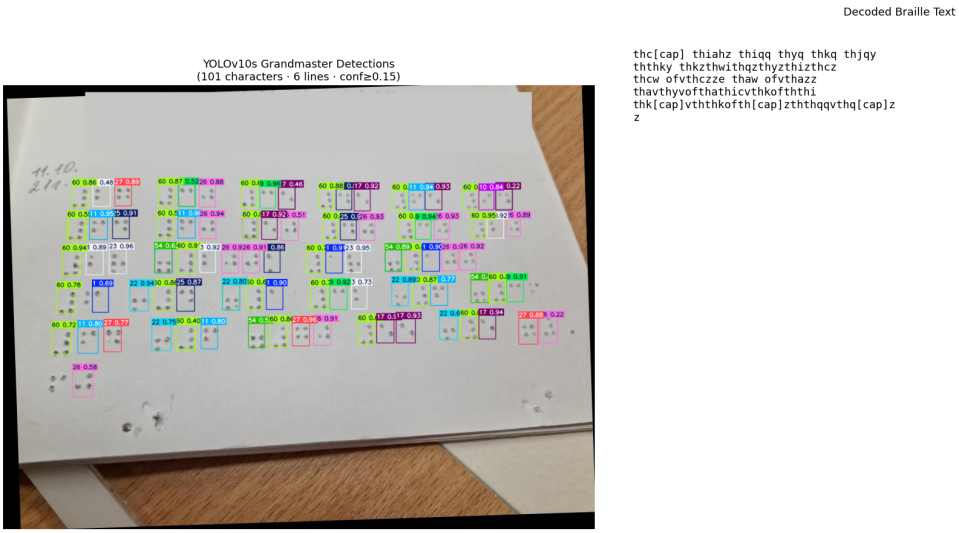

Figure 10: YOLOv10s Grandmaster: end-to-end Braille OCR inference on a full scanned page. Detected characters annotated with bounding boxes; decoded Braille text extracted across 6 lines, 101 characters detected.

Phase 2 — YOLOv8s Results (Youssef Adel)

Table 7: YOLOv8s — Best Validation Results (32 images, 9,139 instances)

Run	mAP@0.50	mAP@0.50:95	Precision	Recall
YOLOv8n 640 px (early run)	0.672	0.527	0.866	0.656
YOLOv8s 1024 px + synthetic	**0.731**	0.576	0.928	0.724

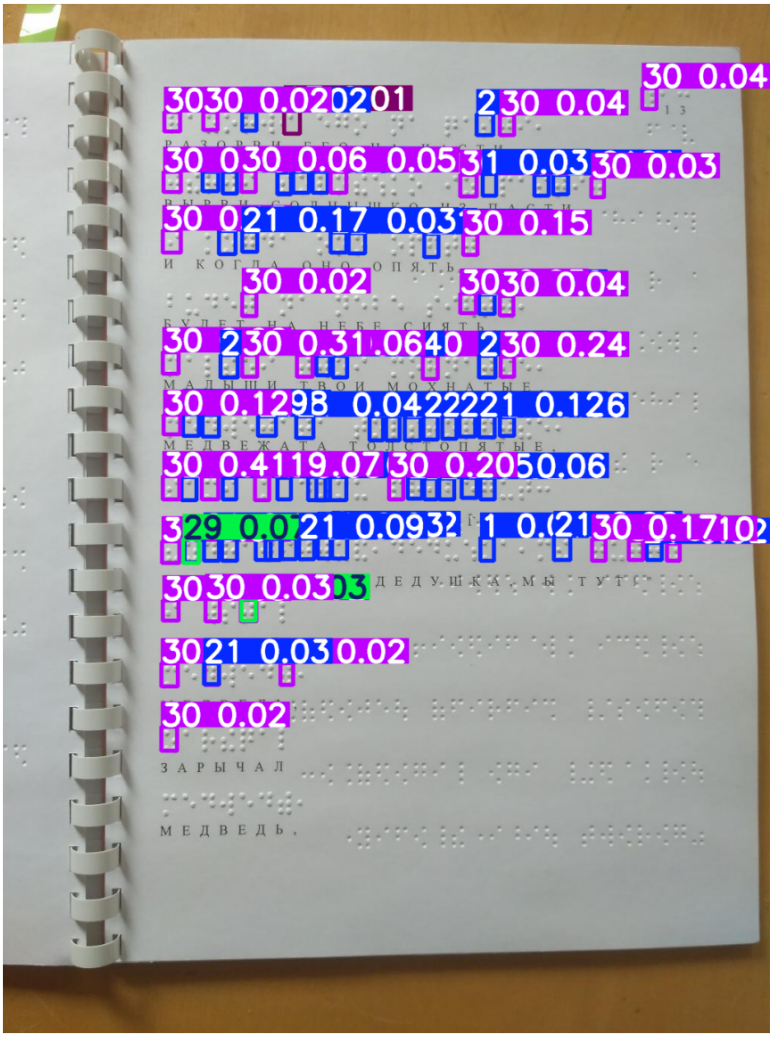

Figure 11: YOLOv8s inference on a full-page Braille scan: detected characters with bounding boxes and class labels. The model successfully localises densely-packed dot patterns.

Phase 2 — YOLO11 Family Results (Mario Nader)

Table 8: YOLO11 Family — Validation Set (32 images, 9,139 instances, 63 classes)

Model	Precision	Recall	mAP@0.50	mAP@0.50:95
YOLO11n (Nano)	0.234	0.116	0.0917	0.0691
YOLO11s (Small)	0.879	0.726	0.724	0.579
YOLO11m (Medium)	**0.914**	**0.776**	**0.776**	**0.677**

For YOLO11m: 16 classes ≥80% AP; 41 classes ≥50% AP; 6 classes <50% AP (‘k’, ‘l’, ‘ou’, ‘ing’, ‘and’, ‘q’).

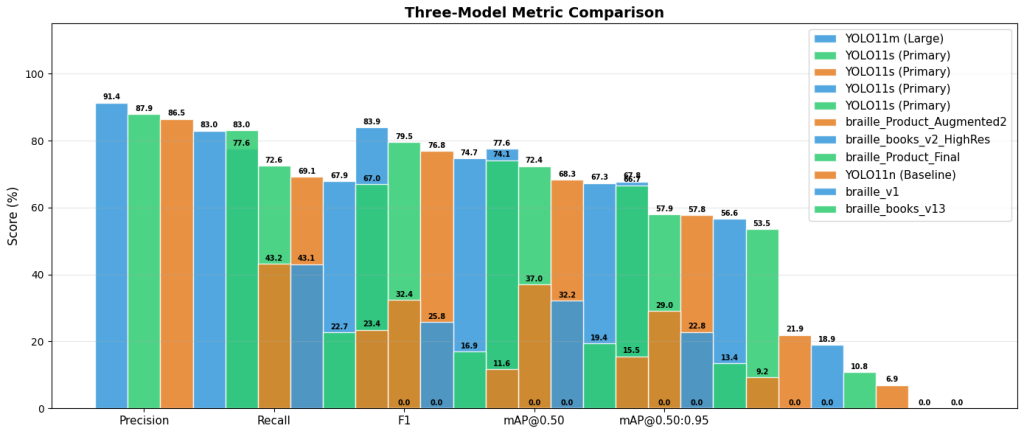

Figure 12: YOLO11 family comparison chart across all 5 metrics. Clear monotonic improvement with model scale on mAP@0.50, Precision, and Recall. YOLO11n is insufficient for 63-class detection at this resolution.

Overall Comparison Table

Table 9: All Models — Final Performance Summary

Model	Task	Primary Metric	Value
SVM (RBF)	26-class cell classification	F1 (weighted)	0.8028
Logistic Regression	26-class cell classification	F1 (weighted)	0.7518
Random Forest	26-class cell classification	F1 (weighted)	0.7423
KNN ($k = 5$)	26-class cell classification	F1 (weighted)	0.6560
Gaussian NB	26-class cell classification	F1 (weighted)	0.6513
Custom CNN v2	52-class crop classification	Accuracy	99.72%
YOLOv8s (1024 px + synthetic)	63-class full-page detection	mAP@0.50	73.1%
YOLOv10s Grandmaster	63-class full-page detection	mAP@0.50	**98.12%**
YOLO11n (Nano)	63-class full-page detection	mAP@0.50	9.17%
YOLO11s (Small)	63-class full-page detection	mAP@0.50	72.4%
YOLO11m (Medium)	63-class full-page detection	mAP@0.50	77.6%

Comparative Discussion

Best performing model. YOLOv10s Grandmaster achieved the highest mAP@0.50 of **98.12%** with only one class (':') below 50% AP and 0/63 classes at zero AP. The NMS-free dual-assignment architecture combined with 200 epochs of AdamW training, wide patience (50 epochs), and targeted augmentation (copy-paste 50%) were the key contributors.

Why YOLOv10s outperformed YOLO11. Despite YOLO11 being a newer ar-

chitecture, YOLOv10s benefited from a significantly longer training budget (200 vs. 100 epochs) and more aggressive augmentation (copy-paste 50% vs. 30%), giving minority classes more opportunity to converge. The NMS-free inference also reduces duplicate detections on dense Braille lines.

Why deep learning outperformed classical ML. Phase 1 operated on isolated, clean 28×28 images — a constrained 26-class classification problem where HOG+SVM is near-optimal. Phase 2 models must simultaneously localise and classify characters on noisy full-page scans at high resolution, requiring spatial reasoning fundamentally beyond global feature classifiers.

Impact of resolution. The upgrade from 640 to 1024 px was the single most impactful change: Braille cells at 640 px shrink below the detection floor. YOLOv8s improved from baseline mAP 67.2% (640 px) to 73.1% (1024 px), and all YOLO variants required 1024 px to produce meaningful detection rates.

8 Error Analysis and Critical Reflection

Misclassification Analysis

Phase 1. SVM errors concentrated between characters differing by a single dot (e.g., 'c' vs. 'd', 'e' vs. 'i'). Gaussian NB showed widespread misclassification due to violated independence assumptions over correlated HOG features.

Phase 2 — CNN. Errors predominantly in classes with low inter-class Hamming distance in dot configuration and in the rarest classes with insufficient training crops. Misclassified samples had consistently lower confidence scores, indicating calibrated uncertainty.

Phase 2 — YOLOv10s. Only 1/63 classes (':') fell below 50% AP@0.50 at 49.50%. This class has a physically small bounding box and few training instances, making it the hardest detection target. All 62 other classes scored above 88% AP. The model successfully recovered all classes that scored 0% AP in a less optimised baseline run.

Phase 2 — YOLO11. Per-class AP analysis (YOLO11m) identified 6 classes below 50% AP: 'k', 'l', 'ou', 'ing', 'and', 'q' — all rare contraction or visually ambiguous characters with fewer training instances.

Confusion Matrix Analysis

The YOLOv10s confusion matrix (Figure 9) shows overwhelming diagonal dominance across all 63 classes with only one visible off-diagonal concentration (class 41, ':'). The

CNN confusion matrix (Figure 7) shows near-perfect diagonal at the 52-class level. The Phase 1 SVM confusion matrix (Figure 4) shows strong per-class accuracy with minor confusion between adjacent-dot characters.

Project Limitations

- **Class imbalance:** Despite copy-paste (50%) and cls weighting (3.0), class 41 (':') remained below 50% AP in YOLOv10s. The 5,296× imbalance is a fundamental data constraint.
- **Missing classes:** 8 of 63 classes have zero training instances in the Angelina books split and cannot be detected by any model trained on this data.
- **Computational constraints:** Batch size limited to 4 at `imgsz=1024`. Extended hyperparameter search was not feasible within Kaggle session limits.
- **Language scope:** Angelina covers English Braille only; Arabic Braille (relevant to Egyptian deployment) is absent.
- **Generalisation unknown:** Models trained and evaluated on the same Angelina dataset family; robustness to different embossers, printers, or scanning conditions is untested.

Possible Improvements

- **More training data:** Annotating additional Braille pages, especially for the 8 missing and low-AP classes, is the highest-impact improvement.
- **WBF ensemble:** Combining YOLOv8s, YOLOv10s, and YOLO11s predictions via Weighted Box Fusion would likely push mAP above 98% without retraining.
- **Larger YOLO variants:** YOLO11l/x provide more capacity for the rarest class features.
- **Braille-specific augmentation:** Simulating embossing defects (faded dots, dot merging) would improve scanner robustness.
- **Language model integration:** A CTC decoder or Braille n-gram model would enforce linguistic consistency in decoded sequences and correct the remaining misclassifications.

9 Conclusion and Future Work

Main Findings:

- Classical ML (SVM, Random Forest) with HOG+PCA achieved strong F1 (0.8028) on 26-class isolated cell classification, confirming that well-engineered features are competitive for constrained tasks.
- For full-page Braille OCR, YOLO-family detectors are essential. **YOLOv10s Grandmaster achieved the best result of mAP@0.50 = 98.12%** (P: 97.16%, R: 97.87%, F1: 97.51%), with 0/63 classes at zero AP and only one class below 50% AP.
- Resolution is the single most impactful hyperparameter: upgrading from 640 to 1024 px is essential for reliable Braille cell detection.
- The Custom CNN achieved 99.72% accuracy on isolated 32×32 crops — a 360K-parameter lightweight architecture sufficient when upstream segmentation is available.
- Class imbalance (ratio up to 5,296×) requires deliberate mitigation (copy-paste, cls weighting, extended patience), but one class (':') remained challenging even for the best model.

Best Performing Models:

- **Full-page detection:** YOLOv10s Grandmaster — mAP@0.50: 98.12%, F1: 97.51%
- **Isolated crop classification:** Custom CNN — Test Accuracy: 99.72%
- **Best within YOLO11 family:** YOLO11m — mAP@0.50: 77.62%

Key Lessons: Data quality and representation dominate model performance. The resolution upgrade contributed more than any architecture change. Modular incremental experimentation (baseline → resolution → augmentation → optimiser) enables clean attribution of gains.

Future Work: Arabic Braille support for Egyptian accessibility applications; WBF ensemble combining all YOLO families; Braille language model integration for contextual decoding; mobile application deployment for real-time camera-based Braille translation.

10 Teamwork and Task Delegation

Team Member	Student ID	Assigned Role	Main Responsibilities	Deliverables Contributed
David Nashaat	247687	ML Lead / Coordinator	Custom CNN architecture design, crop extraction from Angelina dataset, training pipeline, class-weight balancing, experiment tracking, and project coordination	Model 4 (Custom CNN v2): architecture, training, evaluation, confusion matrix, prediction visualisations
Youssef Adel	247121	Data Engineer & ML Engineer	YOLOv8 iterative training; YAML configuration; synthetic data generation for minority classes; `cls=2.0` loss tuning; LLM post-processing pipeline (Groq API, LLaMA-3.3-70B) integration; EDA	Model 5 (YOLOv8s): full training pipeline, mAP evaluation, inference demo with Braille text decoding, dataset EDA
David Nashaat & Youssef Adel	247687 / 247121	Evaluation & Error Analysis Lead	Evaluation strategy design, metric selection (mAP, F1, ROC-AUC), confusion matrix analysis, per-class error analysis, cross-model comparison	Evaluation strategy and error analysis sections, results comparison tables
Abanoub Elwy	243505	ML Engineer	YOLOv10s Grandmaster: full production pipeline (single model), hyperparameter justification table, EDA (56,994 annotations, 5296× imbalance), Kaggle-based training, per-class AP analysis (0/63 zero-AP), end-to-end Braille OCR inference pipeline with Gradio deployment	Model 6 (YOLOv10s Grandmaster): complete training and evaluation pipeline, EDA, confusion matrix, per-class AP, OCR inference, Gradio app
Mario Nader	248655	ML Engineer	YOLO11 family comparative study (nano/small/medium), shared hyperparameter design, per-class AP analysis framework, model comparison charting, YOLO11 training and evaluation	Model 7 (YOLO11n/s/m): all three variants, evaluation metrics, per-class AP charts, model comparison report

Table 10: Teamwork and Task Delegation

11 Appendix

Phase 1 — Full Preprocessing Pipeline

```
Input: 1,560 Braille images (28x28 grayscale, JPEG)
Step 1: Load as grayscale, resize to 28x28
Step 2: Otsu's Thresholding --> binary (dots=255, bg=0)
```

```
Step 3: Normalise to [0.0, 1.0]
Step 4: HOG extraction --> 1,152-dim vector
Step 5: Label Encoding (a-z --> 0-25)
Step 6: Stratified 80/20 split (seed=42)  --> 1248 train, 312 test
Step 7: StandardScaler (fit on X_train only)
Step 8: PCA 95% variance --> 139 components
Output: X_train (1248x139), X_test (312x139)
```

Phase 2 — Custom CNN Architecture

```
Input: (32, 32, 1) grayscale crops
Block 1-3: Conv2D(32/64/128, 3x3)x2 --> BN --> ReLU
           --> MaxPool(2x2) --> Dropout(0.25/0.25/0.30)
Head:    GlobalAveragePooling2D
         Dense(256, relu, L2=1e-4) --> Dropout(0.40)
         Dense(128, relu, L2=1e-4) --> Dropout(0.30)
         Dense(52, softmax, float32)

Optimizer: Adam(lr=5e-4) | Loss: Categorical Crossentropy
Total Parameters: 360,852 | Best Val Accuracy: 99.75%
Test Accuracy: 99.72%     | Test F1-Score (w): 0.9972
```

Phase 2 — YOLO Training Configuration

Table 11: YOLO Model Training Configuration Summary

Model	imgsz	Epochs	Batch	Optimizer	Patience
YOLOv8n (Youssef v1)	640	50	8	SGD (auto)	100
YOLOv8s (Youssef final)	1024	100+	4	SGD	100
YOLOv10s Grandmaster (Abanoub)	1024	200	4	AdamW	50
YOLO11n (Mario)	1024	100	4	Auto	15
YOLO11s (Mario)	1024	100	4	Auto	15
YOLO11m (Mario)	1024	100	4	Auto	15

Braille Class Mapping (Key Classes)

Classes 0–25: letters a–z. Classes 26–35: digits 1–0. Classes 36–62: capital indicator, number indicator, period, comma, question, exclamation, hyphen, apostrophe, and contractions: *and* (51), *for* (52), *of* (53), *the* (54), *with* (55), *ch* (56), *gh* (57), *sh* (58), *th*

(59), *wh* (60), *ed* (61), *er* (62).

References

[1] I. G. Ovodov, "Optical braille recognition using object detection neural network," in *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, pp. 1741–1748, 2021. Dataset available at `https://github.com/IlyaOvodov/AngelinaDataset`.

[2] F. Agustina, E. H. Rachmawanto, N. K. D. A. Putri, F. R. Saputro, H. Lestiawan, Suprayogi, and S. Huda, "Gabor wavelet and multiclass support vector machine for braille image classification," *Journal of Soft Computing Exploration*, vol. 5, no. 2, 2024.

[3] S. Sana, S. Riaz, S. S. Rizvi, *et al.*, "Deep learning scheme for character prediction with position-free touch screen-based braille input method," *Human-centric Computing and Information Sciences*, vol. 10, no. 41, 2020.

[4] S. Shokat, S. Riaz, S. S. Rizvi, *et al.*, "Characterization of english braille patterns using automated tools and rica-based feature extraction methods," *Sensors*, vol. 22, no. 5, p. 1836, 2022.